

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I OMKAR ESWAR K (192311431) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the

presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. Smart Banking and Financial Management System

A financial institution develops a Smart Banking and Financial Management System to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the Factory Pattern to create different account types, the Strategy Pattern for loan-interest and payment calculation policies, and the Observer Pattern for transaction and fraud alerts. A Repository Pattern is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

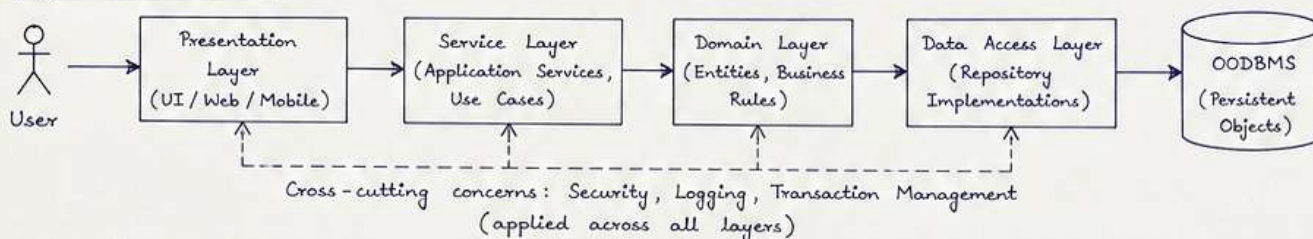
Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

Ans: Role of OOP principles:

- Encapsulation - customer, account, loan, transaction etc. are modeled as classes hiding internal data and exposing controlled operations.
- Abstraction - interfaces hide implementation details (e.g., PaymentService, LoanPolicy).
- Inheritance - common features in Account, Loan, Customer classes are reused and specialized for different types.
- Polymorphism - different behaviors (interest calculation, payment process) are handled through a common interface.

Layered Architecture:



Design Patterns Used:

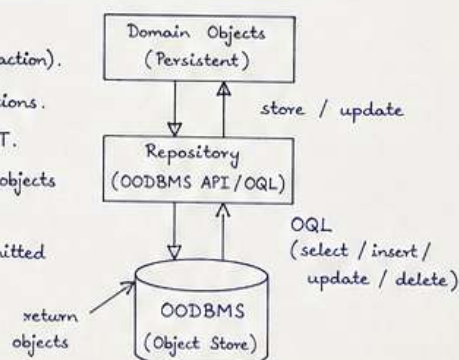
- 1) Factory Pattern - creates different account objects (Savings, Current, FixedDeposit).
- 2) Strategy Pattern - different loan interest calculation and payment policies can be plugged in at runtime.
- 3) Observer Pattern - transaction events (debit, credit, large transfer) notify registered observers for SMS, Email, or Fraud Alert.
- 4) Repository Pattern - abstracts data access logic and manages persistent banking objects.

Contribution to Key Qualities:

Scalability	Layered architecture separates concerns, allowing independent scaling of UI, services, or data layer. New services (investment, wallets) can be added easily.
Security	Security checks are centralized in service/domain layers. Sensitive data access is restricted to data access layer through repositories and OODBMS security.
Flexibility	Patterns like Factory and Strategy allow new account types or calculation policies without changing existing code.
Maintainability	High cohesion and low coupling via layers and patterns make the system easy to test, modify, and extend.

Communication with OODBMS:

- Application objects in the domain layer are persistent (e.g., Account, Customer, Transaction).
- Repositories in the data access layer use OODBMS APIs/OQL to perform operations.
- To store: Repository calls OODBMS.makePersistent(object) or uses OQL INSERT.
- To retrieve: Repository executes OQL SELECT queries and the OODBMS returns objects which are reconstituted into the application.
- Updates: Changes in objects are tracked by the OODBMS and flushed/committed as a transaction.



2. Smart Retail and Inventory Management System

A retail organization develops a Smart Retail and Inventory Management System to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the MVC Pattern for managing user interfaces, the Factory Pattern for creating different product categories, and the Observer Pattern to notify managers when inventory reaches a minimum threshold. The DAO Pattern handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

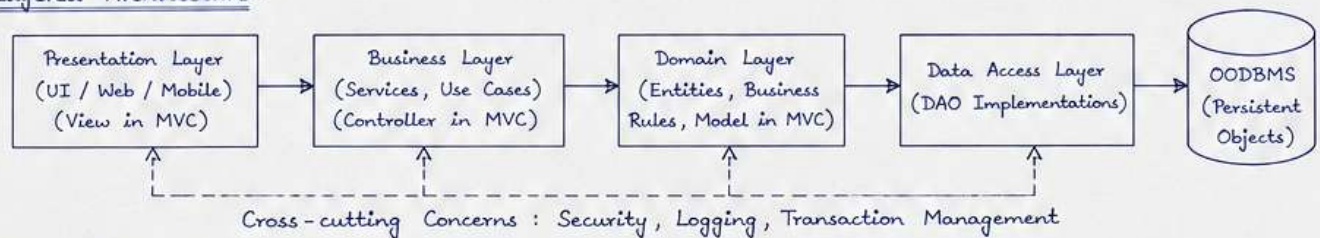
Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

Ans: Role of OOP Principles:

- Encapsulation — Product, Inventory, Order, Customer etc. are modeled as classes hiding data and exposing controlled operations.
- Abstraction — Interfaces (e.g., InventoryService, ProductDAO) hide implementation details.
- Inheritance — Common attributes and behavior are inherited by specialized product types.
- Polymorphism — Different product/category behaviors or pricing strategies are handled through a common interface.

Layered Architecture:



Design Patterns Used:

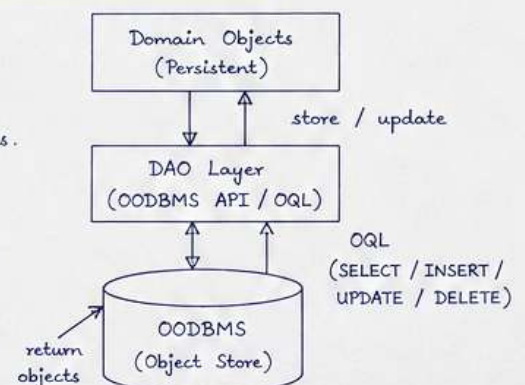
- 1) MVC Pattern — separates data (Model), UI (View) and control (Controller) for better separation and parallel development.
- 2) Factory Pattern — creates different product category objects (Electronics, Clothing, Groceries etc.) without exposing concrete classes to client.
- 3) Observer Pattern — inventory observers (Managers, Notification services) are notified when stock goes below minimum level.
- 4) DAO Pattern — provides a clean interface for CRUD operations on persistent inventory objects and separates business logic from data access logic.

Contribution to Key Qualities:

Quality	How Achieved
Flexibility	Design patterns make it easy to add new product categories, new notification channels, or replace data access mechanism without changing other layers.
Scalability	Layered architecture enables independent scaling (e.g., scale business services separately, use caching in data access layer, distribute OODBMS).
Maintainability	Separation of concerns, low coupling and high cohesion simplifies testing, modification and extension of the system.

Interaction of Layers with OODBMS:

- Domain objects (Product, Inventory, Order, Stock, Supplier etc.) are persistent and mapped to OODBMS object classes.
- DAO layer uses OODBMS APIs/Query Language (OQL) to perform operations.
- To Store : DAO.save(object) uses OODBMS.makePersistent(object) or OQL INSERT.
- To Retrieve : DAO.find(criteria) executes OQL SELECT queries; OODBMS returns persistent objects which are reconstituted.
- Updates : Changes made to persistent objects are tracked by OODBMS and saved on commit (flush).



3. Smart Hotel and Hospitality Management System

A hotel chain develops a Smart Hotel Management System to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the Strategy Pattern for different pricing and discount policies, the Factory Pattern for creating room and service objects, and the Observer Pattern for reservation and service notifications. A Repository Pattern is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

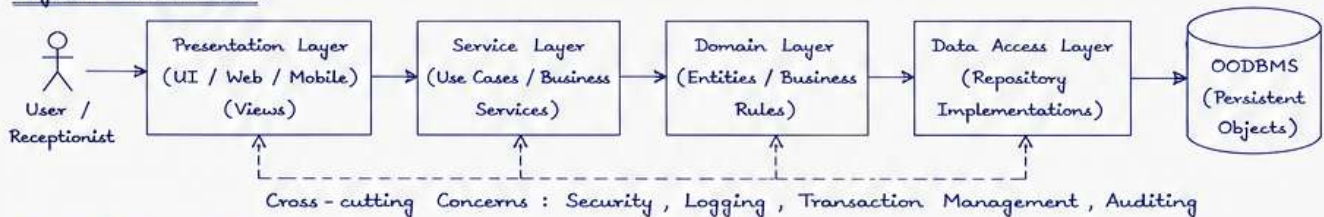
Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

Ams: Role of OOP Principles:

- Encapsulation – Guest, Room, Reservation, Payment, Service etc. are modeled as classes hiding internal data and exposing well-defined operations.
- Abstraction – Interfaces (e.g., PaymentService, PricingPolicy, RoomService) hide implementation details and provide clean contracts.
- Inheritance – Common features (e.g., Room) are inherited by specialized room types (StandardRoom, DeluxeRoom, SuiteRoom).
- Polymorphism – Different pricing/discount policies or service behaviors are used interchangeably through a common interface.

Layered Architecture:



Design Patterns Used:

- 1) Strategy Pattern – different pricing and discount policies (e.g., RegularRate, WeekendRate, MembershipDiscount) can be plugged in at runtime.
- 2) Factory Pattern – creates different room types (StandardRoom, DeluxeRoom, SuiteRoom) and service objects (Laundry, FoodService, Housekeeping) without exposing creation logic.
- 3) Observer Pattern – reservation status changes or service requests notify interested objects (Housekeeping, Billing, NotificationService) automatically.
- 4) Repository Pattern – provides an abstraction over OODBMS operations and isolates business logic from data access details.

Contribution to Key Qualities:

Quality	How Achieved
Flexibility	Strategy Pattern allows new pricing/discount policies to be added without changing existing code. Factory Pattern helps to add new rooms/services easily.
Reusability	Common interfaces and base classes enable reuse of services, rooms, and policies in different modules and future systems.
Maintainability	Separation of concerns through layers and patterns makes the code easy to understand, test, extend and modify.

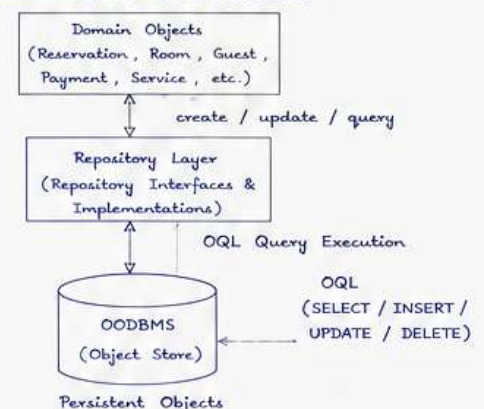
How Layers Coordinate Reservation Processing:

1. Guest makes a reservation request through UI.
2. Presentation Layer collects data and sends request to Service Layer.
3. Service Layer validates request, applies business rules and uses Strategy object for pricing/discount calculation.
4. Service Layer uses Domain objects (Guest, Room, Reservation, Payment) to create or update reservation details.
5. Service Layer calls Repository interface to save or fetch persistent objects.
6. Confirmation is sent back to UI and notifications are triggered through Observer objects.

Benefits to Future Enhancements:

- New smart-room automation or personalized services can be added as new services and observers without affecting existing modules.
- Loose coupling between layers and use of patterns ease integration with external systems and future technologies.

Communication with OODBMS:



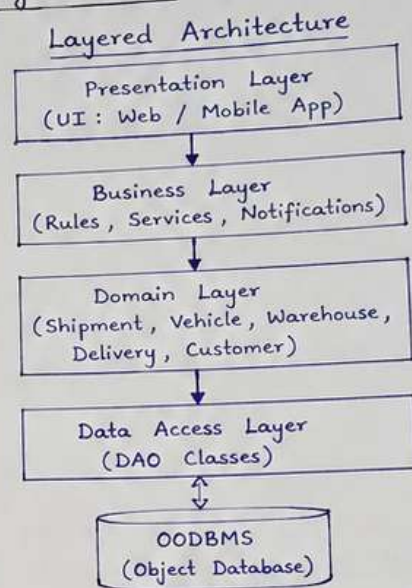
4. Smart Logistics and Delivery Management System

A logistics company develops a Smart Logistics and Delivery Management System to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the Strategy Pattern for selecting optimal delivery routes, the Factory Pattern for creating different shipment types, and the Observer Pattern for real-time delivery status notifications. A DAO Pattern manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

How OOP Principles, Layered Architecture and Design Patterns help

- Encapsulation keeps shipment, vehicle and delivery data safe within classes and hides internal details.
- Inheritance allows common features to be shared among different types of shipments, vehicles and users, reducing duplication.
- Polymorphism enables different delivery methods and route-selection strategies to work through a common interface.
- Abstraction hides complex implementation and exposes only necessary features.
- These principles make the system modular, reusable, flexible and easy to maintain.



Role of Design Patterns

- Strategy Pattern : Different route algorithms (shortest, fastest, AI-based) can be used interchangeably.
- Factory Pattern : Creates different shipment objects (Standard, Fragile, Express, Bulk) based on type.
- Observer Pattern : Notifies customers and other components in real-time when delivery status changes.
- DAO Pattern : Separates business logic from data access logic, which improves maintainability and testability.

How Layers Communicate with OODBMS

1. User requests are received in the Presentation Layer.
2. Requests are passed to the Business Layer, where validation, rules and processes are applied.
3. Business Layer uses Domain Layer objects to perform operations.
4. Data Access Layer (DAO) converts objects to persistent form and sends requests to the OODBMS to store, retrieve, update or delete shipment, vehicle and delivery data.
5. OODBMS executes the operation and returns results.
6. DAO receives the results, converts them into domain objects and sends them back to the Business Layer.
7. Finally, the response is returned to the Presentation Layer and shown to the user.

Benefits

- Adaptability : New shipment types, vehicles or route algorithms can be added without changing existing code.
- Scalability : Each layer can be scaled independently to handle more users and data.
- Maintainability : Clear separation of concerns and use of design patterns makes the system easier to test, modify and extend.

Thus, object-oriented design, layered architecture and design patterns together build a flexible, scalable and maintainable Smart Logistics and Delivery Management System.

5. Smart Energy Management System

An energy management company develops a Smart Energy Management System to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules and the data access layer communicates with an OODBMS. The system uses the Observer Pattern for consumption alerts, the Strategy Pattern for different energy optimization algorithms, and the Factory Pattern for creating different smart-meter objects. A Repository Pattern is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Role of OOP Principles

- **Encapsulation** : Energy data, meter details and user information are kept within classes, protecting them from unauthorized access.
- **Inheritance** : Common features of meters, users and energy devices are reused, reducing code duplication.
- **Polymorphism** : Different optimization algorithms and alert mechanisms can be used through a common interface.
- **Abstraction** : Complex energy calculation and database operations are hidden from users, exposing only necessary functions.

Role of Design Patterns

- **Observer Pattern** : Observers (users, admins, mobile apps) are notified instantly when abnormal consumption is detected.
- **Strategy Pattern** : Different energy optimization algorithms (cost based, efficiency based, AI based) can be selected at runtime.
- **Factory Pattern** : Creates different types of smart meters (prepaid, postpaid, industrial, domestic) without changing client code.
- **Repository Pattern** : Provides an abstraction over OODBMS to perform CRUD operations on persistent energy objects.

How Layers Interact with OODBMS

1. The user interacts with the system through dashboards in the Presentation Layer.
2. The Presentation Layer sends requests to the Business Layer (e.g., view energy usage, set alert).
3. The Business Layer applies rules, performs analysis and calls domain objects to process the request.
4. The Domain Layer represents core entities such as SmartMeter, EnergyReading, User, Tariff, Alert.
5. The Business Layer requests the Data Access Layer (Repository) to store or retrieve data.
6. The Repository Layer communicates with the OODBMS using its query interface and performs operations like insert, update, delete or retrieve.
7. The OODBMS stores persistent energy objects and returns results to the Repository, which sends them back to the Business Layer and finally to the Presentation Layer.

Scalability, Extensibility and Maintainability

- **Scalability** : New meters, users or optimization algorithms can be added without affecting existing components.
- **Extensibility** : New renewable-energy modules or AI prediction engines can be integrated by extending domain classes and implementing new strategies.
- **Maintainability** : Separation of concerns through layers and patterns makes the system easy to test, modify and maintain.

Thus, object-oriented principles, layered architecture and design patterns together make the Smart Energy Management System scalable, extensible, reliable and easy to maintain.

Layered Architecture

